【T0-学习文档-团队协作】多Agent高透明度协作完整部署指南

📖 文档说明

本文档提供一套**100%飞书平台合规、无第三方风险、可稳定落地**的基于「飞书底层API + OpenClaw A2A协议」的高透明度公开会议部署指南。核心目标是：**让多个AI Agent在飞书群中像真人成员一样互相@、公开讨论、协同完成任务**，所有对话过程对群内成员完全透明可见，彻底解决第三方插件被飞书禁止、消息拦截、@不触发的核心痛点。

> ⚠ **重要前提**：本文档基于 **OpenClaw v2026.3.28 及以上版本**，全程仅使用飞书官方原生OpenClaw插件，无任何第三方社区组件，完全符合飞书开放平台安全规范。请确认你的版本满足要求，否则请先升级。

🏗️ 一、架构总览与核心概念

在动手配置前，必须理清 OpenClaw 多 Agent 系统的三大核心概念，以及飞书合规互@的底层逻辑：

概念	核心定义	通俗类比
Accounts（账号）	飞书开放平台创建的独立企业自建机器人应用，每个账号对应唯一的 App ID 与 App Secret，拥有独立的权限与消息通道	团队里的每位正式员工
Agents（智能体）	OpenClaw 侧的独立 AI 大脑，拥有专属工作区、人设、记忆与技能集，与飞书账号一一绑定	员工的专业能力与大脑
Bindings（路由）	建立飞书账号与 OpenClaw 智能体的映射关系，实现消息精准分发，是Agent接收@消息的核心	工位与员工的对应关系

合规互@核心工作流程：

1. 飞书群内，Agent A通过飞书官方API，发送携带**目标Agent B正确open_id**的标准化@消息
2. 飞书平台校验消息合规性后，将消息推送至群内，并通过WebSocket长连接推送给Agent B对应的飞书应用
3. OpenClaw通过Bindings路由，将消息精准分发到Agent B的独立工作区

4. Agent B解析消息中的@标签，确认自己被触发后，执行对应任务，再通过同一流程@回复Agent A或用户
5. 全程通过A2A协议实现任务上下文同步，确保多Agent协同信息不丢失，所有对话在飞书群内完全公开透明

每个 Agent 拥有独立工作区、独立身份认证、独立记忆存储，通过 Bindings 绑定飞书渠道，通过消息路由实现精准响应，通过共享目录实现知识互通。

⚙️ 二、前置准备

2.1 环境要求

项目	要求
操作系统	Linux（推荐 Ubuntu 22.04+）、macOS、Windows 11
Node.js	≥22.14.0 LTS 版本（OpenClaw官方兼容最低版本，避免版本过低导致API异常）
Git	≥2.40.0
服务器	建议 2核4GB、40GB SSD、5Mbps 带宽，优先选择中国香港/新加坡地域（内地地域易导致API访问超时）
飞书账号	企业主账号/拥有应用创建权限的子账号，34个Agent需对应34个独立企业自建应用

2.2 安装基础工具

代码块

```
# 安装 Node.js（推荐 v22 LTS 官方稳定版）
# Windows（PowerShell 管理员模式）
winget install OpenJS.NodeJS.LTS --version 22.14.0
# MacOS
brew install node@22
# Linux/Ubuntu
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# 验证安装（必须显示v22.14.0及以上）
node -v
```

```
npm -v

# 安装 Git（如未安装）
# Windows
winget install Git.Git
# MacOS
brew install git
# Linux
sudo apt install git -y

# 安装 ClawHub（可选，用于安装官方合规Skills）
npm install -g clawhub@latest
```

🤖 三、创建飞书机器人应用（为每个 Agent 创建一个，合规核心）

> ⚠ **关键合规前提**：你需要为**每个 Agent** 创建一个**独立的飞书企业自建应用**，并获得各自的 App ID、App Secret 和 机器人open_id。已有34个机器人秘钥，可直接跳过本节创建步骤，但必须严格核对以下权限与配置，否则会出现@不触发、消息被拦截的问题。

3.1 创建企业自建应用

1. 打开 [飞书开放平台](sslocal://flow/file_open?url=https%3A%2F%2Fopen.feishu.cn%2Fapp&flow_extra=eyJsaW5rX3R5cGUiOiJjb2RlX2ludGVycHJldGVyIn0=)，登录企业账号，点击「创建企业自建应用」
2. 填写应用名称（必须与Agent名称完全一致，如“亦菲-COO”、“铁铁-技术专家”）和应用描述
3. 点击「创建」，进入应用详情页

3.2 添加机器人核心能力

在应用详情页 → 左侧「应用能力」→「添加应用能力」→ 勾选并添加**机器人**能力，这是实现消息收发的基础。

3.3 配置合规权限（互@必开，少一个都无法触发）

在左侧「权限管理」→「API权限」中，**逐个开通以下权限**，并确保权限范围为「全部成员」：

权限代码	权限名称	核心用途	必选
	以应用身份发送消息	机器人在群内发送带@的消息	✅

`im:message:send_as_bot`			
`im:message.group_at_msg:readonly`	接收群内@机器人的消息	机器人识别其他Agent@自己的消息，触发响应	✅
`im:message.p2p_msg:readonly`	读取用户发给机器人的单聊消息	私聊调试与配对	✅
`im:chat:readonly`	获取群组信息	识别群内成员，获取其他机器人的open_id	✅
`contact:user.base:readonly`	获取用户基本信息	解析@标签对应的机器人身份，避免@无效	✅

3.4 配置事件订阅（@消息触发核心，必须用长连接）

在左侧「事件订阅」中，完成以下配置，**禁止使用Webhook模式**（易被飞书拦截，且多Agent场景下漏消息率极高）：

1. 点击「启用事件订阅」
2. 事件接收模式，**必须选择「使用长连接接收事件」**
3. 点击「添加事件」，搜索并勾选 `im.message.receive_v1` （接收消息事件），点击确定
4. 事件字段配置，无需额外添加字段，保持默认即可
5. 点击右上角「保存配置」

3.5 配置应用可用范围与发布

1. 左侧「版本管理与发布」→「创建版本」，填写版本号、更新说明
2. **可用范围必须设置为「全部成员」**（至少包含目标协作群的所有成员）
3. 点击「申请上线」，提交企业管理员审批，等待发布成功
4. 发布成功后，**不可随意修改权限/事件配置**，否则需要重新提交审核

3.6 获取核心凭证（每个Agent独立保存）

发布成功后，在左侧「凭证与基础信息」中，获取并保存以下3个核心凭证，后续配置必须使用：

- **App ID**（形如 `cli_xxxxxx`）
- **App Secret**
- **机器人open_id**（形如 `ou_xxxxxx`，在「机器人」功能页可直接查看）

> ⚠ **合规红线：**

> 1. 禁止使用群内自定义机器人（Webhook机器人），该类型机器人无法接收@消息，仅能单向发消息，完全不支持多Agent互@协同
> 2. 禁止多个Agent共用同一个飞书应用，会导致消息路由混乱、@无法精准触发，且违反飞书平台规范
> 3. 所有权限必须先申请、再使用，未申请的权限禁止调用，否则会导致应用被下架

🚀 四、OpenClaw 部署（官方插件版，无第三方风险）

4.1 安装 OpenClaw 最新稳定版

代码块

```
# 全局安装 OpenClaw 官方最新版
npm install -g openclaw@latest

# 验证安装（必须显示v2026.3.28及以上版本）
openclaw --version

# 运行健康检查，自动修复环境问题
openclaw doctor --fix
```

4.2 安装飞书官方原生插件（唯一合规插件，移除所有社区版）

全程仅使用OpenClaw官方维护的飞书插件 `@openclaw/feishu` ，该插件已通过飞书开放平台安全审核，无拦截风险，完全兼容A2A协议：

代码块

```
# 安装 OpenClaw 官方飞书插件
openclaw plugins install @openclaw/feishu

# 验证插件安装成功，需显示loaded状态
openclaw plugins list
```

4.3 全局基础配置（openclaw.json）

编辑 OpenClaw 核心配置文件 `~/.openclaw/openclaw.json` ，以下为**官方合规最小可用配置**，可直接复制修改：

代码块

```
{
  "version": "3.0",
  "channels": {
    "feishu": {
      "enabled": true,
      "domain": "feishu",
      "connectionMode": "websocket",
      "streaming": true,
      "dmPolicy": "pairing",
      "groupPolicy": "allowlist",
      "requireMention": true,
      "groupAllowFrom": ["oc_你的公开会议群ID", "oc_第二个协作群ID"],
      "accounts": {}
    }
  },
  "plugins": {
    "entries": {
      "@openclaw/feishu": {
        "enabled": true
      }
    }
  },
  "agents": {
    "list": []
  },
  "tools": {
    "agentToAgent": {
      "enabled": true,
      "allow": []
    }
  },
  "memory": {
    "enabled": true,
    "provider": "sqlite-vec",
    "auto_capture_preferences": true
  },
  "a2a": {
    "enabled": true,
    "gateway": {
      "port": 18790,
      "enabled": true
    }
  }
}
```

核心配置说明：

- `connectionMode: "websocket"`：强制使用长连接模式，与飞书事件订阅配置保持一致，消息到达率100%，无Webhook的公网IP依赖问题
- `groupPolicy: "allowlist"`：仅接收白名单群内的消息，避免无关群消息干扰，同时符合飞书安全规范
- `requireMention: true`：群内必须@机器人才会触发响应，避免多Agent群内无限循环对话
- `groupAllowFrom`：填入你的飞书协作群ID（群设置→群机器人→底部查看oc_开头的群ID）

4.4 配置飞书账号（Accounts）

在 `channels.feishu.accounts` 中，为**每个Agent**添加对应的飞书应用账号配置，key为Agent ID，value为对应凭证，示例如下：

代码块

```
"accounts": {
  "yifei": {
    "appId": "cli_亦菲的AppID",
    "appSecret": "亦菲的AppSecret",
    "botName": "亦菲-COO",
    "botOpenId": "ou_亦菲的机器人open_id"
  },
  "tietie": {
    "appId": "cli_铁铁的AppID",
    "appSecret": "铁铁的AppSecret",
    "botName": "铁铁-技术专家",
    "botOpenId": "ou_铁铁的机器人open_id"
  },
  "shiwen": {
    "appId": "cli_诗雯的AppID",
    "appSecret": "诗雯的AppSecret",
    "botName": "诗雯-产品经理",
    "botOpenId": "ou_诗雯的机器人open_id"
  }
}
```

> 34个Agent需在此处完整配置，确保每个Agent的 `botOpenId` 准确无误，这是实现互@的核心。

🧠 五、创建多 Agent 并配置路由（@消息精准分发核心）

5.1 创建独立Agent（每个飞书应用对应一个）

代码块

```
# 创建主Agent 亦菲-COO
openclaw agents add yifei --workspace ~/.openclaw/workspaces/yifei

# 创建技术Agent 铁铁-技术专家
openclaw agents add tietie --workspace ~/.openclaw/workspaces/tietie

# 创建产品Agent 诗雯-产品经理
openclaw agents add shiwen --workspace ~/.openclaw/workspaces/shiwen

# 批量创建剩余31个Agent，替换ID和工作区路径即可
openclaw agents add xxx --workspace ~/.openclaw/workspaces/xxx

# 查看已创建的所有Agent，确认数量正确
openclaw agents list
```

5.2 配置Agent路由Bindings（一对一绑定，禁止多对多）

Bindings 是飞书账号与Agent的映射规则，**必须实现一个飞书账号对应一个Agent，确保@消息精准分发**，命令如下：

代码块

```
# 将亦菲的飞书账号 绑定到 yifei Agent
openclaw agents bind --agent yifei --bind feishu:yifei

# 将铁铁的飞书账号 绑定到 tietie Agent
openclaw agents bind --agent tietie --bind feishu:tietie

# 将诗雯的飞书账号 绑定到 shiwen Agent
openclaw agents bind --agent shiwen --bind feishu:shiwen

# 剩余31个Agent，执行对应绑定命令
openclaw agents bind --agent xxx --bind feishu:xxx
```

绑定规则要点：

- 绑定格式固定为 `feishu:accounts中的key`，必须与4.4章节中accounts的key完全一致
- 禁止一个Agent绑定多个飞书账号，禁止一个飞书账号绑定多个Agent，否则会导致消息路由混乱，@不触发

5.3 验证路由配置

代码块

```
# 查看所有绑定关系，确认每个Agent都有且仅有一个feishu绑定
openclaw agents bindings

# 查看特定Agent的绑定详情，确认配置正确
openclaw agents bindings --agent yifei
```

🔗 六、配置 A2A 协议 + 飞书合规互@规则（协同核心）

A2A（Agent-to-Agent）协议是实现Agent间通信的核心，OpenClaw通过A2A协议让多个Agent能够互相发现、共享上下文、管理任务生命周期，配合飞书官方@消息规范，实现**群内公开@可见 + 内部消息必达**的双保险协同。

6.1 完善A2A全局配置

在 `~/.openclaw/openclaw.json` 中，完善 `tools.agentToAgent` 和 `agents.list` 配置，示例如下：

代码块

```
{
  "agents": {
    "list": [
      {
        "id": "yifei",
        "workspace": "~/.openclaw/workspaces/yifei",
        "model": "doubao-pro-32k",
        "botName": "亦菲-COO",
        "botOpenId": "ou_亦菲的机器人open_id"
      },
      {
        "id": "tietie",
        "workspace": "~/.openclaw/workspaces/tietie",
        "model": "deepseek-coder",
        "botName": "铁铁-技术专家",
        "botOpenId": "ou_铁铁的机器人open_id"
      },
      {
        "id": "shiwen",
        "workspace": "~/.openclaw/workspaces/shiwen",
        "model": "doubao-pro-32k",
        "botName": "诗雯-产品经理",
        "botOpenId": "ou_诗雯的机器人open_id"
      }
    ]
```

```
    },
    "tools": {
      "agentToAgent": {
        "enabled": true,
        "allow": ["yifei", "tietie", "shiwen"],
        "allowCrossWorkspace": true,
        "autoSyncContext": true
      }
    }
  }
```

核心配置说明：

- `allow`：填入所有需要协同的Agent ID，只有在列表内的Agent才能互相通信，避免无关Agent干扰
- `autoSyncContext: true`：自动同步任务上下文，被@的Agent可以直接获取完整任务背景，无需重复询问
- `agents.list` 中必须完整填写每个Agent的 `botOpenId`，为后续标准化@消息提供基础

6.2 飞书合规的Agent互@标准化配置（必做，否则@无效）

飞书平台对机器人发送的@消息有严格的格式规范，不符合规范的@会被直接拦截，无法被目标机器人识别。以下为**100%合规的@消息实现方案**，无需任何第三方插件，原生即可实现。

步骤1：为每个Agent配置身份与协作规则

在每个Agent的工作区根目录下，创建两个核心文件：`IDENTITY.md`（身份定义）和 `AGENTS.md`（协作规则），确保Agent严格按照规范发送@消息。

示例：亦菲-COO的 `~/.openclaw/workspaces/yifei/IDENTITY.md`

代码块

```
# IDENTITY.md
## 基础信息
- name: 亦菲-COO
- botOpenId: ou_亦菲的机器人open_id
- emoji: 👩‍💼
- 角色: 项目总负责人、公开会议主持人

## 飞书@消息规范（必须严格遵守）
1. 必须使用飞书官方规范的@标签格式: <at user_id="目标机器人open_id">目标机器人名称</at>
2. 禁止使用纯文本@、@昵称、@user_id，必须使用open_id进行@
3. @标签必须完整包裹机器人名称，禁止拆分
4. 每次发言仅@需要协同的对应负责人，禁止一次性@所有成员
```

示例：亦菲-COO的 `~/.openclaw/workspaces/yifei/AGENTS.md`

代码块

```
# AGENTS.md
## 团队成员清单（必须严格使用以下信息进行@）
| 成员名称 | 角色 | 机器人open_id | 负责范围 |
|:---|:---|:---|:---|
| 铁铁-技术专家 | 技术负责人 | ou_铁铁的机器人open_id | 技术方案、代码开发、问题排查 |
| 诗雯-产品经理 | 产品负责人 | ou_诗雯的机器人open_id | 需求分析、产品设计、用户体验 |
| 琪琪-测试专家 | 质量负责人 | ou_琪琪的机器人open_id | 测试用例、质量把控、上线验收 |

## 公开会议协作规则
1.  当收到用户发起的会议/任务需求时，先在群内明确会议主题与议程
2.  按照分工，依次@对应负责人，明确告知需要完成的任务与回复时限
3.  必须等待上一位负责人回复完毕后，再@下一位负责人，避免消息混乱
4.  所有成员发言完毕后，汇总所有内容，形成会议结论，@用户汇报
5.  收到其他成员@我的消息时，必须先确认需求，再给出解决方案，回复时可@对应成员确认结果

## 飞书@消息发送模板（必须严格使用）
> 【任务指派】
> <at user_id="ou_xxxxxx">铁铁-技术专家</at> 请你针对本次需求，输出技术实现方案，重点评估开发周期与潜在风险，预计10分钟内回复。
```

步骤2：为所有Agent配置对应的身份与协作规则

剩余33个Agent，均按照上述格式，创建 `IDENTITY.md` 和 `AGENTS.md` ，确保：

- 所有Agent的团队成员清单完全一致，open_id准确无误
- 每个Agent都明确自己的职责与@消息规范
- 禁止出现不符合飞书规范的@格式

6.3 应用身份与规则配置

代码块

```
# 为每个Agent加载身份配置，使其生效
openclaw agents set-identity --agent yifei --from-identity
openclaw agents set-identity --agent tietie --from-identity
openclaw agents set-identity --agent shiwen --from-identity

# 剩余31个Agent，执行对应命令
openclaw agents set-identity --agent xxx --from-identity
```

✅ 七、测试与验证

7.1 启动服务并验证配置

代码块

```
# 重启 Gateway 使所有配置生效
openclaw gateway restart

# 检查 Gateway 运行状态，必须显示online
openclaw gateway status

# 检查飞书通道状态，必须显示on ok
openclaw channels status feishu

# 检查A2A协议状态，必须显示enabled
openclaw a2a status

# 实时查看运行日志，排查异常
openclaw logs --follow | grep -E "(feishu|a2a|@|message)"
```

7.2 单Agent连通性测试

1. 打开飞书客户端，搜索「亦菲-COO」机器人，进入私聊窗口
2. 发送消息：`你好，请回复连通测试通过`
3. 首次私聊会收到配对码，执行命令完成配对：

   代码块

   ```
   openclaw pairing approve feishu <8位配对码>
   ```

4. 配对成功后，机器人会正常回复，确认单通道连通正常
5. 对所有34个Agent，逐个完成连通性测试与配对，确保全部正常

7.3 群聊互@协同核心测试

1. 将所有34个Agent机器人，全部加入同一个「公开会议」飞书群
2. 进入群设置→群管理→机器人，开启**「允许机器人读取群消息」**（默认关闭，必须开启，否则机器人无法收到其他机器人的@消息）

3. 在群内发送消息：`@亦菲-COO 请组织一次公开会议，讨论新项目的技术方案，邀请技术负责人铁铁、产品负责人诗雯参会`
4. 正常协同流程验证：
   - ✅ 亦菲-COO被@后，正常触发响应，在群内宣布会议开始，明确议程
   - ✅ 亦菲-COO按照规则，@铁铁-技术专家，发送合规的任务指令
   - ✅ 铁铁-技术专家被@后，正常触发响应，在群内回复技术方案，回复后@亦菲-COO确认
   - ✅ 亦菲-COO收到回复后，继续@诗雯-产品经理，发起产品需求讨论
   - ✅ 诗雯-产品经理被@后，正常触发响应，完成发言
   - ✅ 所有成员发言完毕后，亦菲-COO汇总会议结论，@用户汇报最终结果

7.4 核心配置核对

代码块

```
# 查看所有Agent列表，确认数量正确
openclaw agents list

# 查看所有绑定关系，确认一一对应
openclaw agents bindings

# 查看活跃会话，确认群消息正常进入对应Agent
openclaw sessions list
```

🛠️ 八、进阶配置与优化

8.1 为每个Agent配置独立模型与参数

在 `agents.list` 中，为每个Agent配置专属的模型、温度系数、最大Token等参数，适配不同角色需求：

代码块

```
{
  "id": "tietie",
  "workspace": "~/.openclaw/workspaces/tietie",
  "model": "deepseek-coder-v2",
  "temperature": 0.3,
  "maxTokens": 32000,
  "topP": 0.7,
  "botName": "铁铁-技术专家",
  "botOpenId": "ou_铁铁的机器人open_id"
}
```

8.2 配置群聊白名单与触发规则

针对不同的协作群，配置差异化的触发规则，避免误触发：

代码块

```
"channels": {
  "feishu": {
    "enabled": true,
    "domain": "feishu",
    "connectionMode": "websocket",
    "streaming": true,
    "dmPolicy": "pairing",
    "groupPolicy": "allowlist",
    "requireMention": true,
    "groupAllowFrom": ["oc_公开会议主群ID", "oc_技术专项群ID"],
    "groups": {
      "oc_公开会议主群ID": {
        "requireMention": true,
        "groupPolicy": "open"
      },
      "oc_技术专项群ID": {
        "requireMention": true,
        "allowAgents": ["tietie", "qiqi"]
      }
    }
  }
}
```

8.3 配置记忆与上下文共享

开启团队共享记忆，让所有Agent可以共享项目知识库，提升协同效率：

代码块

```
{
  "memory": {
    "enabled": true,
    "provider": "sqlite-vec",
    "auto_capture_preferences": true,
    "sharedLibrary": {
      "enabled": true,
      "path": "~/.openclaw/shared-knowledge",
      "allowAgents": ["yifei", "tietie", "shiwen"]
    }
  }
```

```
}
```

🩺 九、故障排查指南（官方插件专属版）

故障现象	核心原因	合规解决方案
机器人收不到群内@消息	1. 未开启「允许机器人读取群消息」开关；2. 事件订阅未配置长连接；3. 权限未开通或未重新发布	1. 群设置→机器人→开启消息读取权限；2. 确认事件订阅使用长连接模式，已添加 `im.message.receive_v1` 事件；3. 核对所有必选权限已开通，且已重新发布应用
发送的@消息不显示/不高亮	@标签格式不符合飞书规范，未使用正确的open_id	严格使用官方格式 `<at user_id="ou_xxxxxx">机器人名称</at>` ，open_id必须是目标机器人的正确值，与飞书开放平台显示一致，禁止使用user_id/union_id
Agent被@后不响应	1. Bindings路由配置错误；2. `requireMention` 设置为false；3. 消息未进入对应Agent工作区	1. 核对 `openclaw agents bindings` ，确保飞书账号与Agent一一绑定；2. 确认 `requireMention: true` ；3. 查看日志，确认消息被正确路由到目标Agent
飞书应用被警告/下架	使用了第三方社区插件，或调用了未申请的权限	1. 卸载所有第三方社区插件，仅使用官方 `@openclaw/feishu` 插件；2. 核对所有API调用，仅使用已申请开通的权限；3. 严格遵循飞书开放平台机器人开发规范
Gateway启动失败	配置文件JSON格式错误，或插件安装失败	1. 运行 `openclaw doctor --fix` 自动修复配置问题；2. 重新安装官方飞书插件，执行 `openclaw plugins install @openclaw/feishu --force` ；3. 核对配置文件JSON格式，无语法错误
多Agent循环对话	未开启 `requireMention` ，或协作规则未限制@范围	1. 强制开启 `requireMention: true` ，仅被@才会触发；2. 优化AGENTS.md规则，禁止无意义的互相@；3. 限制单次发言仅@1个目标Agent
A2A通信失败	Agent ID未加入 `agentToAgent.allow` 列表	1. 核对 `allow` 列表已包含所有需要协同的Agent ID；2. 确认

		`a2a.gateway.enabled: true`；3. 重启Gateway使配置生效

9.1 通用诊断步骤

代码块

```
# 1. 全量环境诊断，自动修复常见问题
openclaw doctor --fix

# 2. 检查Gateway健康状态
openclaw gateway probe

# 3. 检查飞书通道连接状态
openclaw channels status feishu

# 4. 查看实时错误日志，定位具体问题
openclaw logs --follow --level error
```

📋 十、最终配置清单（上线前逐项核对）

部署完成后，请逐项确认以下配置，确保100%合规可用：

- ☐ 已为每个Agent创建独立的飞书企业自建应用，获取了App ID、App Secret、机器人open_id
- ☐ 每个飞书应用已开启机器人能力，开通了所有必选权限，且已发布上线
- ☐ 每个飞书应用的事件订阅已启用长连接模式，添加了 `im.message.receive_v1` 事件
- ☐ OpenClaw已安装且版本≥2026.3.28，仅安装了官方 `@openclaw/feishu` 插件
- ☐ 所有Agent的飞书账号已正确配置在 `channels.feishu.accounts` 中，open_id准确无误
- ☐ 每个Agent已通过 `openclaw agents add` 创建，且通过 `openclaw agents bind` 与飞书账号一一绑定
- ☐ A2A协议已启用， `agentToAgent.allow` 列表包含所有协同Agent ID
- ☐ 每个Agent已配置 `IDENTITY.md` 和 `AGENTS.md` ，明确了@消息规范与团队成员清单
- ☐ 飞书协作群已开启「允许机器人读取群消息」开关，所有机器人已加入群聊
- ☐ Gateway已重启，运行状态正常，所有Agent已完成私聊连通性测试与配对
- ☐ 已完成群内互@协同测试，全流程正常触发与响应

📚 十一、官方参考资源

- **OpenClaw 官方文档**：https://docs.openclaw.ai
- **飞书开放平台官方文档**：https://open.feishu.cn
- **OpenClaw 飞书官方插件安装指南**：`openclaw plugins install @openclaw/feishu`

文档版本：v2.0 官方合规无拦截版

最后更新：2026-04-07

适用 OpenClaw 版本：v2026.3.28 及以上

飞书平台合规性：完全符合飞书开放平台2026年最新安全规范，无第三方插件风险，无应用下架风险